



07. PyTorch Experiment Tracking Exercise Solutions

Welcome to the 07. PyTorch Experiment Tracking exercise solutions notebook.

Note: There may be more than one solution to each of the exercises. This notebook only shows one possible example.

Resources

1. These exercises/solutions are based on [section 07. PyTorch Transfer Learning](#) of the Learn PyTorch for Deep Learning course by Zero to Mastery.
2. See a live [walkthrough of the solutions \(errors and all\) on YouTube](#).
3. See [other solutions on the course GitHub](#).

Get various imports and helper functions

```
# For this notebook to run with updated APIs, we need torch 1.12+ and torchvision 0.13+
try:
    import torch
    import torchvision
    assert int(torch.__version__.split(".")[1]) >= 12, "torch version should be 1.12+"
    assert int(torchvision.__version__.split(".")[1]) >= 13, "torchvision version should be 0.13+"
    print(f"torch version: {torch.__version__}")
    print(f"torchvision version: {torchvision.__version__}")
except:
    print(f"[INFO] torch/torchvision versions not as required, installing nightly versions.")
    !pip3 install -U --pre torch torchvision --extra-index-url https://download.pytorch.org/whl/nightly/cu113
    import torch
    import torchvision
    print(f"torch version: {torch.__version__}")
    print(f"torchvision version: {torchvision.__version__}")

[INFO] torch/torchvision versions not as required, installing nightly versions.
Looking in indexes: https://pypi.org/simple, https://download.pytorch.org/whl/nightly/cu113
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.9.1)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.24.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.20.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.1)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from torch) (11.3.3.83)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from torch) (10.3.9.90)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from torch) (11.7.3.90)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.5.8.93)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.3.20 in /usr/local/lib/python3.12/dist-packages (from torch) (3.3.20)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.13.1.3)
Requirement already satisfied: triton==3.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.5.1)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision) (2.0.2)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torchvision) (10.4.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
torch version: 2.9.1+cu128
torchvision version: 0.24.1+cu128
```

```
# Make sure we have a GPU
device = "cuda" if torch.cuda.is_available() else "cpu"
device
```

```
'cuda'
```

```
# Get regular imports
import matplotlib.pyplot as plt
import torch
import torchvision

from torch import nn
from torchvision import transforms

# Try to get torchinfo, install it if it doesn't work
try:
    from torchinfo import summary
except:
    print("[INFO] Couldn't find torchinfo... installing it.")
    !pip install -q torchinfo
    from torchinfo import summary

# Try to import the going_modular directory, download it from GitHub if it doesn't work
try:
    from going_modular.going_modular import data_setup, engine
except:
    # Get the going_modular scripts
    print("[INFO] Couldn't find going_modular scripts... downloading them from GitHub.")
    !git clone https://github.com/mrdbourke/pytorch-deep-learning
    !mv pytorch-deep-learning/going_modular .
    !rm -rf pytorch-deep-learning
    from going_modular.going_modular import data_setup, engine
```

```
# Getting setup
import torch
import torchvision

# This notebook requires torch v0.12+ and torchvision v0.13+
print(torch.__version__)
print(torchvision.__version__)
```

```
2.9.1+cu128
0.24.1+cu128
```

```
# Set seeds
def set_seeds(seed: int=42):
    """Sets random sets for torch operations.

    Args:
        seed (int, optional): Random seed to set. Defaults to 42.
    """
    # Set the seed for general torch operations
    torch.manual_seed(seed)
    # Set the seed for CUDA torch operations (ones that happen on the GPU)
    torch.cuda.manual_seed(seed)
```

```
import os
import zipfile

from pathlib import Path

import requests

def download_data(source: str,
                  destination: str,
                  remove_source: bool = True) -> Path:
    """Downloads a zipped dataset from source and unzips to destination.

    Args:
        source (str): A link to a zipped file containing data.
        destination (str): A target directory to unzip data to.
        remove_source (bool): Whether to remove the source after downloading and extracting.

    Returns:
        pathlib.Path to downloaded data.

    Example usage:
        download_data(source="https://github.com/mrdbourke/pytorch-deep-learning/raw/main/data/pizza_steak_sushi.zip
```

```

        destination="pizza_steak_sushi")

    """
    # Setup path to data folder
    data_path = Path("data/")
    image_path = data_path / destination

    # If the image folder doesn't exist, download it and prepare it...
    if image_path.is_dir():
        print(f"[INFO] {image_path} directory exists, skipping download.")
    else:
        print(f"[INFO] Did not find {image_path} directory, creating one...")
        image_path.mkdir(parents=True, exist_ok=True)

    # Download pizza, steak, sushi data
    target_file = Path(source).name
    with open(data_path / target_file, "wb") as f:
        request = requests.get(source)
        print(f"[INFO] Downloading {target_file} from {source}...")
        f.write(request.content)

    # Unzip pizza, steak, sushi data
    with zipfile.ZipFile(data_path / target_file, "r") as zip_ref:
        print(f"[INFO] Unzipping {target_file} data...")
        zip_ref.extractall(image_path)

    # Remove .zip file
    if remove_source:
        os.remove(data_path / target_file)

    return image_path

image_path = download_data(source="https://github.com/mrdbourke/pytorch-deep-learning/raw/main/data/pizza_steak_sush
                           destination="pizza_steak_sushi")
image_path

```

```

[INFO] data/pizza_steak_sushi directory exists, skipping download.
PosixPath('data/pizza_steak_sushi')

```

```

from torch.utils.tensorboard import SummaryWriter
def create_writer(experiment_name: str,
                  model_name: str,
                  extra: str=None):
    """Creates a torch.utils.tensorboard.writer.SummaryWriter() instance saving to a specific log_dir.

    log_dir is a combination of runs/timestamp/experiment_name/model_name/extra.

    Where timestamp is the current date in YYYY-MM-DD format.

    Args:
        experiment_name (str): Name of experiment.
        model_name (str): Name of model.
        extra (str, optional): Anything extra to add to the directory. Defaults to None.

    Returns:
        torch.utils.tensorboard.writer.SummaryWriter(): Instance of a writer saving to log_dir.

    Example usage:
        # Create a writer saving to "runs/2022-06-04/data_10_percent/effnetb2/5_epochs/"
        writer = create_writer(experiment_name="data_10_percent",
                               model_name="effnetb2",
                               extra="5_epochs")

        # The above is the same as:
        writer = SummaryWriter(log_dir="runs/2022-06-04/data_10_percent/effnetb2/5_epochs/")
    """
    from datetime import datetime
    import os

    # Get timestamp of current date (all experiments on certain day live in same folder)
    timestamp = datetime.now().strftime("%Y-%m-%d") # returns current date in YYYY-MM-DD format

    if extra:
        # Create log directory path
        log_dir = os.path.join("runs", timestamp, experiment_name, model_name, extra)
    else:
        log_dir = os.path.join("runs", timestamp, experiment_name, model_name)

```

```
print(f"[INFO] Created SummaryWriter, saving to: {log_dir}...")
return SummaryWriter(log_dir=log_dir)
```

```
# Create a test writer
writer = create_writer(experiment_name="test_experiment_name",
                      model_name="this_is_the_model_name",
                      extra="add_a_little_extra_if_you_want")
```

```
[INFO] Created SummaryWriter, saving to: runs/2025-12-30/test_experiment_name/this_is_the_model_name/add_a_little_ext
```

```
from typing import Dict, List
from tqdm.auto import tqdm

from going_modular.going_modular.engine import train_step, test_step

# Add writer parameter to train()
def train(model: torch.nn.Module,
          train_dataloader: torch.utils.data.DataLoader,
          test_dataloader: torch.utils.data.DataLoader,
          optimizer: torch.optim.Optimizer,
          loss_fn: torch.nn.Module,
          epochs: int,
          device: torch.device,
          writer: torch.utils.tensorboard.writer.SummaryWriter # new parameter to take in a writer
          ) -> Dict[str, List]:
    """Trains and tests a PyTorch model.

    Passes a target PyTorch models through train_step() and test_step()
    functions for a number of epochs, training and testing the model
    in the same epoch loop.

    Calculates, prints and stores evaluation metrics throughout.

    Stores metrics to specified writer log_dir if present.

    Args:
        model: A PyTorch model to be trained and tested.
        train_dataloader: A DataLoader instance for the model to be trained on.
        test_dataloader: A DataLoader instance for the model to be tested on.
        optimizer: A PyTorch optimizer to help minimize the loss function.
        loss_fn: A PyTorch loss function to calculate loss on both datasets.
        epochs: An integer indicating how many epochs to train for.
        device: A target device to compute on (e.g. "cuda" or "cpu").
        writer: A SummaryWriter() instance to log model results to.

    Returns:
        A dictionary of training and testing loss as well as training and
        testing accuracy metrics. Each metric has a value in a list for
        each epoch.
        In the form: {train_loss: [...],
                      train_acc: [...],
                      test_loss: [...],
                      test_acc: [...]}
        For example if training for epochs=2:
        {train_loss: [2.0616, 1.0537],
          train_acc: [0.3945, 0.3945],
          test_loss: [1.2641, 1.5706],
          test_acc: [0.3400, 0.2973]}
    """
    # Create empty results dictionary
    results = {"train_loss": [],
              "train_acc": [],
              "test_loss": [],
              "test_acc": []
    }

    # Loop through training and testing steps for a number of epochs
    for epoch in tqdm(range(epochs)):
        train_loss, train_acc = train_step(model=model,
                                           dataloader=train_dataloader,
                                           loss_fn=loss_fn,
                                           optimizer=optimizer,
                                           device=device)
        test_loss, test_acc = test_step(model=model,
                                         dataloader=test_dataloader,
```

```

        loss_fn=loss_fn,
        device=device)

# Print out what's happening
print(
    f"Epoch: {epoch+1} | "
    f"train_loss: {train_loss:.4f} | "
    f"train_acc: {train_acc:.4f} | "
    f"test_loss: {test_loss:.4f} | "
    f"test_acc: {test_acc:.4f}"
)

# Update results dictionary
results["train_loss"].append(train_loss)
results["train_acc"].append(train_acc)
results["test_loss"].append(test_loss)
results["test_acc"].append(test_acc)

### New: Use the writer parameter to track experiments ###
# See if there's a writer, if so, log to it
if writer:
    # Add results to SummaryWriter
    writer.add_scalars(main_tag="Loss",
                      tag_scalar_dict={"train_loss": train_loss,
                                       "test_loss": test_loss},
                      global_step=epoch)
    writer.add_scalars(main_tag="Accuracy",
                      tag_scalar_dict={"train_acc": train_acc,
                                       "test_acc": test_acc},
                      global_step=epoch)

    # Close the writer
    writer.close()
else:
    pass
### End new ###

# Return the filled results at the end of the epochs
return results

```

▼ Download data

Using the same data from https://www.learnpytorch.io/07_pytorch_experiment_tracking/

```

# Download 10 percent and 20 percent training data (if necessary)
data_10_percent_path = download_data(source="https://github.com/mrdbourke/pytorch-deep-learning/raw/main/data/pizza_s",
                                     destination="pizza_steak_sushi")

data_20_percent_path = download_data(source="https://github.com/mrdbourke/pytorch-deep-learning/raw/main/data/pizza_s",
                                     destination="pizza_steak_sushi_20_percent")

```

```

[INFO] data/pizza_steak_sushi directory exists, skipping download.
[INFO] data/pizza_steak_sushi_20_percent directory exists, skipping download.

```

```

# Setup training directory paths
train_dir_10_percent = data_10_percent_path / "train"
train_dir_20_percent = data_20_percent_path / "train"

# Setup testing directory paths (note: use the same test dataset for both to compare the results)
test_dir = data_10_percent_path / "test"

# Check the directories
print(f"Training directory 10%: {train_dir_10_percent}")
print(f"Training directory 20%: {train_dir_20_percent}")
print(f"Testing directory: {test_dir}")

```

```

Training directory 10%: data/pizza_steak_sushi/train
Training directory 20%: data/pizza_steak_sushi_20_percent/train
Testing directory: data/pizza_steak_sushi/test

```

```

from torchvision import transforms

# Create a transform to normalize data distribution to be inline with ImageNet
normalize = transforms.Normalize(mean=[0.485, 0.456, 0.406], # values per colour channel [red, green, blue]

```

```
std=[0.229, 0.224, 0.225])

# Create a transform pipeline
simple_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(), # get image values between 0 & 1
    normalize
])
```

Turn data into DataLoaders

```
BATCH_SIZE = 32

# Create 10% training and test DataLoaders
train_dataloader_10_percent, test_dataloader, class_names = data_setup.create_dataloaders(train_dir=train_dir_10_per
                                                                                          test_dir=test_dir,
                                                                                          transform=simple_transform
                                                                                          batch_size=BATCH_SIZE)

# Create 20% training and test DataLoaders
train_dataloader_20_percent, test_dataloader, class_names = data_setup.create_dataloaders(train_dir=train_dir_20_per
                                                                                          test_dir=test_dir,
                                                                                          transform=simple_transform
                                                                                          batch_size=BATCH_SIZE)

# Find the number of samples/batches per dataloader (using the same test_dataloader for both experiments)
print(f"Number of batches of size {BATCH_SIZE} in 10 percent training data: {len(train_dataloader_10_percent)}")
print(f"Number of batches of size {BATCH_SIZE} in 20 percent training data: {len(train_dataloader_20_percent)}")
print(f"Number of batches of size {BATCH_SIZE} in testing data: {len(test_dataloader)} (all experiments
print(f"Number of classes: {len(class_names)}, class names: {class_names}")

Number of batches of size 32 in 10 percent training data: 8
Number of batches of size 32 in 20 percent training data: 15
Number of batches of size 32 in testing data: 8 (all experiments will use the same test set)
Number of classes: 3, class names: ['pizza', 'steak', 'sushi']
```

Exercise 1: Pick a larger model from [torchvision.models](#) to add to the list of experiments (for example, EffNetB3 or higher)

- How does it perform compared to our existing models?

In notebook 07, we tried out EffNetB0 and EffNetB2 with ~4M parameters and ~9M parameters respectively, let's try step things up by using a bigger model.

Let's try EfficientNet_V2_S (efficientnet version 2 small, EffNetV2_S for short) which has ~20M parameters -

https://pytorch.org/vision/main/models/generated/torchvision.models.efficientnet_v2_s.html#torchvision.models.EfficientNet_V2_S_Weights

To keep things simple, let's just compare EffNetB2 (best performing model) to EffNetV2_S

Create models

```
import torchvision.models as models
effnetv2_s_weights = models.EfficientNet_V2_S_Weights.DEFAULT # .DEFAULT = "best available weights"
effnetv2_s = models.efficientnet_v2_s(weights=effnetv2_s_weights)
# effnetv2_s
```

```
def create_effnetb2(out_features=len(class_names)):
    weights = torchvision.models.EfficientNet_B2_Weights.DEFAULT
    model = torchvision.models.efficientnet_b2(weights=weights).to(device)
    dropout = 0.3
    in_features=1408

    # Freeze the base model layers
    for param in model.features.parameters():
        param.requires_grad=False

    # Set the seeds
    set_seeds()
```

```

# Update the classifier head
model.classifier = nn.Sequential(
    nn.Dropout(p=dropout, inplace=True),
    nn.Linear(in_features=in_features,
              out_features=out_features)
).to(device)

# Set the model name
model.name = "effnetb2"
print(f"[INF0] Creating {model.name} feature extractor model...")
return model

def create_effnetv2_s(out_features=len(class_names)):
    weights = torchvision.models.EfficientNet_V2_S_Weights.DEFAULT
    model = torchvision.models.efficientnet_v2_s(weights=weights).to(device)
    dropout = 0.2
    in_features=1280

    # Freeze the base model layers
    for param in model.features.parameters():
        param.requires_grad=False

    # Set the seeds
    set_seeds()

    # Update the classifier head
    model.classifier = nn.Sequential(
        nn.Dropout(p=dropout, inplace=True),
        nn.Linear(in_features=in_features,
                  out_features=out_features)
    ).to(device)

    # Set the model name
    model.name = "effnetv2_s"
    print(f"[INF0] Creating {model.name} feature extractor model...")
    return model

```

```

from torchinfo import summary

#summary(model=create_effnetv2_s(),
#        input_size=(1, 3, 224, 224),
#        col_names=["input_size", "output_size", "num_params", "trainable"],
#        col_width=20,
#        row_settings=["var_names"])

```

✓ Setup modelling experiments

Going to use similar code to: https://www.learnpytorch.io/07_pytorch_experiment_tracking/#76-create-experiments-and-set-up-training-code

For our modelling experiments, mainly want to substitute EffNetB0 experiments with EffNetV2_S in its place.

```

# Create epoch list
num_epochs = [5, 10]

# Create models list
models = ["effnetb2", "effnetv2_s"]

# Create dataloaders dictionary for various dataloaders
train_dataloaders = {"data_10_percent": train_dataloader_10_percent,
                     "data_20_percent": train_dataloader_20_percent}

```

```

%time
from going_modular.going_modular.utils import save_model

# 1. Set the random seeds
set_seeds(seed=42)

# 2. Keep track of experiment numbers
experiment_number = 0

# 3. Loop through each DataLoader

```

```

for dataloader_name, train_dataloader in train_data loaders.items():

    # 4. Loop through each number of epochs
    for epochs in num_epochs:

        # 5. Loop through each model name and create a new model based on the name
        for model_name in models:

            # 6. Create information print outs
            experiment_number += 1
            print(f"[INFO] Experiment number: {experiment_number}")
            print(f"[INFO] Model: {model_name}")
            print(f"[INFO] DataLoader: {dataloader_name}")
            print(f"[INFO] Number of epochs: {epochs}")

            # 7. Select the model
            if model_name == "effnetb2":
                model = create_effnetb2()
            else:
                model = create_effnetv2_s()

            # 8. Create a new loss and optimizer for every model
            loss_fn = nn.CrossEntropyLoss()
            optimizer = torch.optim.Adam(params=model.parameters(), lr=0.001)

            # 9. Train target model with target dataloaders and track experiments
            train(model=model,
                  train_dataloader=train_dataloader,
                  test_dataloader=test_dataloader,
                  optimizer=optimizer,
                  loss_fn=loss_fn,
                  epochs=epochs,
                  device=device,
                  writer=create_writer(experiment_name=dataloader_name,
                                      model_name=model_name,
                                      extra=f"{epochs}_epochs"))

            # 10. Save the model to file so we can get back the best model
            save_filepath = f"07_{model_name}_{dataloader_name}_{epochs}_epochs.pth"
            save_model(model=model,
                      target_dir="models",
                      model_name=save_filepath)
            print("-"*50 + "\n")

```



```
[INFO] Experiment number: 1
[INFO] Model: effnetb2
[INFO] DataLoader: data_10_percent
[INFO] Number of epochs: 5
Downloading: "https://download.pytorch.org/models/efficientnet_b2_rwightman-c35c1473.pth" to /root/.cache/torch/hub/c
100%|██████████| 35.2M/35.2M [00:00<00:00, 112MB/s] [INFO] Creating effnetb2 feature extractor model...
[INFO] Created SummaryWriter, saving to: runs/2025-12-30/data_10_percent/effnetb2/5_epochs...
```

```
100% 5/5 [00:11<00:00, 2.20s/it]
Epoch: 1 | train_loss: 1.0845 | train_acc: 0.3984 | test_loss: 0.9591 | test_acc: 0.6903
Epoch: 2 | train_loss: 0.9241 | train_acc: 0.6406 | test_loss: 0.8659 | test_acc: 0.8144
Epoch: 3 | train_loss: 0.8330 | train_acc: 0.7344 | test_loss: 0.7461 | test_acc: 0.9384
Epoch: 4 | train_loss: 0.7067 | train_acc: 0.8906 | test_loss: 0.7138 | test_acc: 0.8778
Epoch: 5 | train_loss: 0.6750 | train_acc: 0.7891 | test_loss: 0.7052 | test_acc: 0.8873
[INFO] Saving model to: models/07_effnetb2_data_10_percent_5_epochs.pth
```

```
[INFO] Experiment number: 2
[INFO] Model: effnetv2_s
[INFO] DataLoader: data_10_percent
[INFO] Number of epochs: 5
[INFO] Creating effnetv2_s feature extractor model...
[INFO] Created SummaryWriter, saving to: runs/2025-12-30/data_10_percent/effnetv2_s/5_epochs...
100% 5/5 [00:11<00:00, 2.26s/it]
Epoch: 1 | train_loss: 1.0771 | train_acc: 0.3047 | test_loss: 0.9163 | test_acc: 0.7121
Epoch: 2 | train_loss: 0.9092 | train_acc: 0.6953 | test_loss: 0.8707 | test_acc: 0.6932
Epoch: 3 | train_loss: 0.8490 | train_acc: 0.6523 | test_loss: 0.7619 | test_acc: 0.7652
Epoch: 4 | train_loss: 0.7325 | train_acc: 0.7578 | test_loss: 0.6267 | test_acc: 0.9271
Epoch: 5 | train_loss: 0.6777 | train_acc: 0.8594 | test_loss: 0.6222 | test_acc: 0.8759
[INFO] Saving model to: models/07_effnetv2_s_data_10_percent_5_epochs.pth
```

```
[INFO] Experiment number: 3
[INFO] Model: effnetb2
[INFO] DataLoader: data_10_percent
[INFO] Number of epochs: 10
[INFO] Creating effnetb2 feature extractor model...
[INFO] Created SummaryWriter, saving to: runs/2025-12-30/data_10_percent/effnetb2/10_epochs...
100% 10/10 [00:25<00:00, 3.11s/it]
Epoch: 1 | train_loss: 1.0845 | train_acc: 0.3984 | test_loss: 0.9591 | test_acc: 0.6903
Epoch: 2 | train_loss: 0.9241 | train_acc: 0.6406 | test_loss: 0.8659 | test_acc: 0.8144
Epoch: 3 | train_loss: 0.8330 | train_acc: 0.7344 | test_loss: 0.7461 | test_acc: 0.9384
Epoch: 4 | train_loss: 0.7067 | train_acc: 0.8906 | test_loss: 0.7138 | test_acc: 0.8778
Epoch: 5 | train_loss: 0.6750 | train_acc: 0.7891 | test_loss: 0.7052 | test_acc: 0.8873
Epoch: 6 | train_loss: 0.6031 | train_acc: 0.7773 | test_loss: 0.6310 | test_acc: 0.9280
Epoch: 7 | train_loss: 0.5891 | train_acc: 0.8008 | test_loss: 0.6354 | test_acc: 0.8873
Epoch: 8 | train_loss: 0.5280 | train_acc: 0.9336 | test_loss: 0.6177 | test_acc: 0.8873
Epoch: 9 | train_loss: 0.5382 | train_acc: 0.8125 | test_loss: 0.6268 | test_acc: 0.8466
Epoch: 10 | train_loss: 0.4938 | train_acc: 0.9180 | test_loss: 0.5865 | test_acc: 0.8873
[INFO] Saving model to: models/07_effnetb2_data_10_percent_10_epochs.pth
```

```
[INFO] Experiment number: 4
[INFO] Model: effnetv2_s
[INFO] DataLoader: data_10_percent
[INFO] Number of epochs: 10
[INFO] Creating effnetv2_s feature extractor model...
[INFO] Created SummaryWriter, saving to: runs/2025-12-30/data_10_percent/effnetv2_s/10_epochs...
100% 10/10 [00:23<00:00, 2.41s/it]
Epoch: 1 | train_loss: 1.0771 | train_acc: 0.3047 | test_loss: 0.9163 | test_acc: 0.7121
Epoch: 2 | train_loss: 0.9092 | train_acc: 0.6953 | test_loss: 0.8707 | test_acc: 0.6932
Epoch: 3 | train_loss: 0.8490 | train_acc: 0.6523 | test_loss: 0.7619 | test_acc: 0.7652
Epoch: 4 | train_loss: 0.7325 | train_acc: 0.7578 | test_loss: 0.6267 | test_acc: 0.9271
Epoch: 5 | train_loss: 0.6777 | train_acc: 0.8594 | test_loss: 0.6222 | test_acc: 0.8759
Epoch: 6 | train_loss: 0.5990 | train_acc: 0.8789 | test_loss: 0.6254 | test_acc: 0.9072
Epoch: 7 | train_loss: 0.6350 | train_acc: 0.7422 | test_loss: 0.5644 | test_acc: 0.8864
Epoch: 8 | train_loss: 0.5375 | train_acc: 0.9102 | test_loss: 0.5216 | test_acc: 0.8561
Epoch: 9 | train_loss: 0.5828 | train_acc: 0.7578 | test_loss: 0.5201 | test_acc: 0.8153
Epoch: 10 | train_loss: 0.5122 | train_acc: 0.8047 | test_loss: 0.4969 | test_acc: 0.8864
[INFO] Saving model to: models/07_effnetv2_s_data_10_percent_10_epochs.pth
```

Inspect model results

```
%load_ext tensorboard
%tensorboard --logdir runs
[INFO] DataLoader: data_20_percent
[INFO] Number of epochs: 5
[INFO] Creating effnetb2 feature extractor model...
[INFO] Created SummaryWriter, saving to: runs/2025-12-30/data_20_percent/effnetb2/5_epochs...
100% 5/5 [00:16<00:00, 3.24s/it]
Epoch: 1 | train_loss: 0.9896 | train_acc: 0.5208 | test_loss: 0.7830 | test_acc: 0.8049
```

Epoch: 2 | train_loss: 0.7384 | train_acc: 0.7646 | test_loss: 0.6643 | test_acc: 0.8769

TensorBoard

TIME SERIES

SCALARS

INACTIVE

Filter runs (regex)

Filter tags (regex)

All

Scalars

Image

Histogram

Settings



Run

Pinned

Settings



2025-12-30/data_10_per

Pin cards for a quick view and comparison



2025-12-30/data_10_per

Accuracy



2025-12-30/data_10_per



2025-12-30/data_10_per



2025-12-30/data_10_per



2025-12-30/data_10_per



2025-12-30/data_10_per



2025-12-30/data_10_per



2025-12-30/data_10_per



2025-12-30/data_10_per



2025-12-30/data_10_per

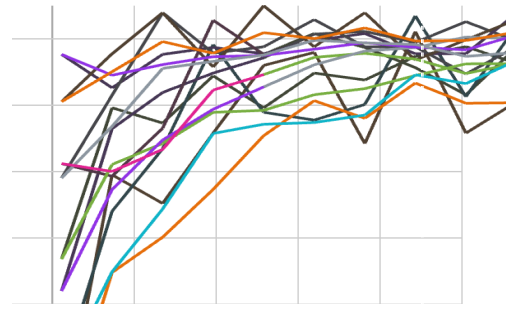


2025-12-30/data_10_per



2025-12-30/data_10_per

Accuracy



Run

2025-12-30/data_10_percent/effnetv2/10_epochs/Accuracy_test_acc
2025-12-30/data_10_percent/effnetv2/10_epochs/Accuracy_train_acc

Smoothed

0.8787

0.8666

Loss

SCALARS

Smoothing

Tooltip sorting method

Alphabetical

☒ Ignore outliers in chart scaling

Partition non-monotonic X axis

HISTOGRAMS

[INFO] Number of epochs: 10

[INFO] Creating effnetv2_s feature extractor model...

[INFO] Created SummaryWriter, saving to: runs/2025-12-30/data_10_percent/effnetv2_s/10_epochs...

EffNetV2_s with 10 epochs and 20% of the data gets the best performance (lowest test loss and highest test accuracy).

See results:

Epoch: 1 | train_loss: 0.9749 | train_acc: 0.5687 | test_loss: 0.7457 | test_acc: 0.8759

Epoch: 2 | train_loss: 0.7311 | train_acc: 0.7658 | test_loss: 0.6436 | test_acc: 0.8769

[INFO] Experiment number: 8

[INFO] Model: effnetv2_s

[INFO] DataLoader: data_20_percent

[INFO] Number of epochs: 10

[INFO] Creating effnetv2_s feature extractor model...

[INFO] Created SummaryWriter, saving to: runs/2022-06-23/data_20_percent/effnetv2_s/10_epochs...

100%

10/10 [00:36<00:00, 3.61s/it]

Epoch: 1 | train_loss: 0.9752 | train_acc: 0.5667 | test_loss: 0.7430 | test_acc: 0.8864

Epoch: 2 | train_loss: 0.7255 | train_acc: 0.7917 | test_loss: 0.6404 | test_acc: 0.8456

Epoch: 3 | train_loss: 0.6232 | train_acc: 0.7708 | test_loss: 0.5183 | test_acc: 0.8759

Epoch: 4 | train_loss: 0.5495 | train_acc: 0.8313 | test_loss: 0.4845 | test_acc: 0.8864

Epoch: 5 | train_loss: 0.5441 | train_acc: 0.7833 | test_loss: 0.4278 | test_acc: 0.8769

Epoch: 6 | train_loss: 0.4487 | train_acc: 0.8479 | test_loss: 0.4083 | test_acc: 0.9072

Epoch: 7 | train_loss: 0.4416 | train_acc: 0.8396 | test_loss: 0.3806 | test_acc: 0.8769

Epoch: 8 | train_loss: 0.4004 | train_acc: 0.8708 | test_loss: 0.3997 | test_acc: 0.8977

Epoch: 9 | train_loss: 0.3747 | train_acc: 0.8917 | test_loss: 0.3980 | test_acc: 0.8873

Epoch: 10 | train_loss: 0.3958 | train_acc: 0.8625 | test_loss: 0.3372 | test_acc: 0.9479

[INFO] Saving model to: models/07_effnetv2_s_data_20_percent_10_epochs.pth

Exercise 2. Introduce data augmentation to the list of experiments using the 20% pizza, steak, sushi training and test datasets, does this change anything?

- For example, you could have one training DataLoader that uses data augmentation (e.g. `train_dataloader_20_percent_aug` and `train_dataloader_20_percent_no_aug`) and then compare the results of two of the same model types training on these two DataLoaders.
- Note:** You may need to alter the `create_dataloaders()` function to be able to take a transform for the training data and the testing data (because you don't need to perform data augmentation on the test data). See [04. PyTorch Custom Datasets section 6](#) for examples of using data augmentation or the script below for an example:

```
# Note: Data augmentation transform like this should only be performed on training data
train_transform_data_aug = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.TrivialAugmentWide(),
    transforms.ToTensor(),
    normalize
])

# Have to update `create_dataloaders()` to handle different augmentations
import os
from torch.utils.data import DataLoader
from torchvision import datasets

NUM_WORKERS = os.cpu_count() # use maximum number of CPUs for workers to load data

# Note: this is an update version of data_setup.create_dataloaders to handle
# differnt train and test transforms.
def create_dataloaders(
    train_dir,
    test_dir,
    train_transform, # add parameter for train transform (transforms on train dataset)
    test_transform, # add parameter for test transform (transforms on test dataset)
    batch_size=32, num_workers=NUM_WORKERS
):
    # Use ImageFolder to create dataset(s)
    train_data = datasets.ImageFolder(train_dir, transform=train_transform)
    test_data = datasets.ImageFolder(test_dir, transform=test_transform)

    # Get class names
    class_names = train_data.classes

    # Turn images into data loaders
    train_dataloader = DataLoader(
        train_data,
        batch_size=batch_size,
        shuffle=True,
        num_workers=num_workers,
        pin_memory=True,
    )
    test_dataloader = DataLoader(
        test_data,
        batch_size=batch_size,
        shuffle=True,
        num_workers=num_workers,
        pin_memory=True,
    )

    return train_dataloader, test_dataloader, class_names
```

Create data augmentation and non-data augmentation transforms

What we want now:

- 2x different training dataloaders:
 1. A 20% DataLoader *with* augmentation (let's use <https://pytorch.org/vision/main/generated/torchvision.transforms.TrivialAugmentWide.html>)
 2. A 20% DataLoader *without* augmentation
- Use EffNetV2_s model
- Train for 5 and 10 epochs

```
# Create a data augmentation transform
from torchvision import transforms

# Create a transform to normalize data distribution to be inline with ImageNet
normalize = transforms.Normalize(mean=[0.485, 0.456, 0.406], # values per colour channel [red, green, blue]
                                std=[0.229, 0.224, 0.225])

data_aug_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.TrivialAugmentWide(),
    transforms.ToTensor(),
    normalize
])

# Create a non-data aug transform
no_data_aug_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    normalize
])
```

✓ Alter `data_setup.create_data_loaders()` to work with multiple transforms

```
# Create dataloaders, let's do this by updating data_setup.create_data_loaders()
import os

from torchvision import datasets, transforms
from torch.utils.data import DataLoader

NUM_WORKERS = os.cpu_count()

def create_data_loaders(
    train_dir: str,
    test_dir: str,
    train_transform: transforms.Compose, ### New (only want to do data augmentation on the training dataset)
    test_transform: transforms.Compose, ### New
    batch_size: int,
    num_workers: int=NUM_WORKERS
):
    """Creates training and testing DataLoaders.

    Takes in a training directory and testing directory path and turns
    them into PyTorch Datasets and then into PyTorch DataLoaders.

    Args:
        train_dir: Path to training directory.
        test_dir: Path to testing directory.
        transform: torchvision transforms to perform on training and testing data.
        batch_size: Number of samples per batch in each of the DataLoaders.
        num_workers: An integer for number of workers per DataLoader.

    Returns:
        A tuple of (train_dataloader, test_dataloader, class_names).
        Where class_names is a list of the target classes.
        Example usage:
        train_dataloader, test_dataloader, class_names = \
            = create_data_loaders(train_dir=path/to/train_dir,
                                test_dir=path/to/test_dir,
                                transform=some_transform,
                                batch_size=32,
                                num_workers=4)
    """
    # Use ImageFolder to create dataset(s)
```

```

train_data = datasets.ImageFolder(train_dir,
                                   transform=train_transform) ### New
test_data = datasets.ImageFolder(test_dir,
                                   transform=test_transform) ### New

# Get class names
class_names = train_data.classes

# Turn images into data loaders
train_dataloader = DataLoader(
    train_data,
    batch_size=batch_size,
    shuffle=True,
    num_workers=num_workers,
    pin_memory=True,
)
test_dataloader = DataLoader(
    test_data,
    batch_size=batch_size,
    shuffle=False,
    num_workers=num_workers,
    pin_memory=True,
)

return train_dataloader, test_dataloader, class_names

```

```

# Create train and test directories
train_20_percent_dir = image_path / "train"
test_20_percent_dir = image_path / "test"

BATCH_SIZE = 32

# Create train dataloader *with* data augmentation
train_dataloader_20_percent_with_aug, test_dataloader_20_percent, class_names = create_dataloaders(train_dir=train_20_percent_dir,
                                                                                                    test_dir=test_20_percent_dir,
                                                                                                    train_transform=train_transform,
                                                                                                    test_transform=test_transform,
                                                                                                    batch_size=BATCH_SIZE)

# Create train dataloader *without* data augmentation
train_dataloader_20_percent_without_aug, test_dataloader_20_percent, class_names = create_dataloaders(train_dir=train_20_percent_dir,
                                                                                                    test_dir=test_20_percent_dir,
                                                                                                    train_transform=None,
                                                                                                    test_transform=test_transform,
                                                                                                    batch_size=BATCH_SIZE)

```

✓ Create a helper function for viewing different DataLoader images

```

# Visualize different samples from both dataloaders (aug and no aug)
def view_dataloader_images(dataloader, n=10):
    if n > 10:
        print(f"Having n higher than 10 will create messy plots, lowering to 10.")
        n = 10
    imgs, labels = next(iter(dataloader))
    plt.figure(figsize=(16, 8))
    for i in range(n):
        # Min max scale the image for display purposes
        targ_image = imgs[i]
        sample_min, sample_max = targ_image.min(), targ_image.max()
        sample_scaled = (targ_image - sample_min) / (sample_max - sample_min)

        # Plot images with appropriate axes information
        plt.subplot(1, 10, i+1)
        plt.imshow(sample_scaled.permute(1, 2, 0)) # resize for Matplotlib requirements
        plt.title(class_names[labels[i]])
        plt.axis(False)

```

```

# Check out samples with data augmentation
view_dataloader_images(train_dataloader_20_percent_with_aug)

```



```
# Checkout samples without data augmentation
view_data_loader_images(train_data_loader_20_percent_without_aug)
```



✓ Run data aug vs no data aug experiments

```
# Setup number of epochs
num_epochs = [5, 10]

# Create dataloaders dictionary for various dataloaders
train_dataloaders = {"data_20_percent_with_aug": train_data_loader_20_percent_with_aug,
                    "data_20_percent_without_aug": train_data_loader_20_percent_without_aug}

# Create model
models = ["effnetv2_s"]
```

```
%%time
from going_modular.going_modular.utils import save_model

# 1. Set the random seeds
set_seeds(seed=42)

# 2. Keep track of experiment numbers
experiment_number = 0

# 3. Loop through each DataLoader
for data_loader_name, train_data_loader in train_dataloaders.items():

    # 4. Loop through each number of epochs
    for epochs in num_epochs:

        # 5. Loop through each model name and create a new model based on the name
        for model_name in models:

            # 6. Create information print outs
            experiment_number += 1
            print(f"[INFO] Experiment number: {experiment_number}")
            print(f"[INFO] Model: {model_name}")
            print(f"[INFO] DataLoader: {data_loader_name}")
            print(f"[INFO] Number of epochs: {epochs}")

            # 7. Select the model
            if model_name == "effnetb2":
                model = create_effnetb2()
            else:
                model = create_effnetv2_s()

            # 8. Create a new loss and optimizer for every model
            loss_fn = nn.CrossEntropyLoss()
            optimizer = torch.optim.Adam(params=model.parameters(), lr=0.001)

            # 9. Train target model with target dataloaders and track experiments
            train(model=model,
                  train_data_loader=train_data_loader,
                  test_data_loader=test_data_loader_20_percent, ### New, use test_data_loader_20_percent
                  optimizer=optimizer,
                  loss_fn=loss_fn,
```

```
epochs=epochs,
device=device,
writer=create_writer(experiment_name=dataloader_name,
                     model_name=model_name,
                     extra=f"{epochs}_epochs"))

# 10. Save the model to file so we can get back the best model
save_filepath = f"07_{model_name}_{dataloader_name}_{epochs}_epochs.pth"
save_model(model=model,
           target_dir="models",
           model_name=save_filepath)
print("-"*50 + "\n")
```



```
[INFO] Experiment number: 1
[INFO] Model: effnetv2_s
[INFO] DataLoader: data_20_percent_with_aug
[INFO] Number of epochs: 5
[INFO] Creating effnetv2_s feature extractor model...
[INFO] Created SummaryWriter, saving to: runs/2025-12-30/data_20_percent_with_aug/effnetv2_s/5_epochs...
100%          5/5 [00:12<00:00, 2.42s/it]
Epoch: 1 | train_loss: 1.0587 | train_acc: 0.3320 | test_loss: 0.8976 | test_acc: 0.6108
Epoch: 2 | train_loss: 0.9347 | train_acc: 0.6367 | test_loss: 0.8542 | test_acc: 0.6723
Epoch: 3 | train_loss: 0.8841 | train_acc: 0.5938 | test_loss: 0.7765 | test_acc: 0.7642
Epoch: 4 | train_loss: 0.8000 | train_acc: 0.7070 | test_loss: 0.6442 | test_acc: 0.8655
Epoch: 5 | train_loss: 0.7197 | train_acc: 0.7383 | test_loss: 0.5972 | test_acc: 0.8655
[INFO] Saving model to: models/07_effnetv2_s_data_20_percent_with_aug_5_epochs.pth
```

```
[INFO] Experiment number: 2
[INFO] Model: effnetv2_s
[INFO] DataLoader: data_20_percent_with_aug
[INFO] Number of epochs: 10
[INFO] Creating effnetv2_s feature extractor model...
[INFO] Created SummaryWriter, saving to: runs/2025-12-30/data_20_percent_with_aug/effnetv2_s/10_epochs...
100%          10/10 [00:24<00:00, 2.51s/it]
Epoch: 1 | train_loss: 1.0587 | train_acc: 0.3320 | test_loss: 0.8976 | test_acc: 0.6108
Epoch: 2 | train_loss: 0.9347 | train_acc: 0.6367 | test_loss: 0.8542 | test_acc: 0.6723
Epoch: 3 | train_loss: 0.8841 | train_acc: 0.5938 | test_loss: 0.7765 | test_acc: 0.7642
Epoch: 4 | train_loss: 0.8000 | train_acc: 0.7070 | test_loss: 0.6442 | test_acc: 0.8655
Epoch: 5 | train_loss: 0.7197 | train_acc: 0.7383 | test_loss: 0.5972 | test_acc: 0.8655
Epoch: 6 | train_loss: 0.6703 | train_acc: 0.8633 | test_loss: 0.5841 | test_acc: 0.8665
Epoch: 7 | train_loss: 0.6599 | train_acc: 0.7500 | test_loss: 0.5479 | test_acc: 0.8665
Epoch: 8 | train_loss: 0.5785 | train_acc: 0.9023 | test_loss: 0.5082 | test_acc: 0.8352
Epoch: 9 | train_loss: 0.6256 | train_acc: 0.7461 | test_loss: 0.5092 | test_acc: 0.8248
Epoch: 10 | train_loss: 0.5756 | train_acc: 0.7852 | test_loss: 0.4693 | test_acc: 0.8352
[INFO] Saving model to: models/07_effnetv2_s_data_20_percent_with_aug_10_epochs.pth
```

```
[INFO] Experiment number: 3
[INFO] Model: effnetv2_s
[INFO] DataLoader: data_20_percent_without_aug
[INFO] Number of epochs: 5
[INFO] Creating effnetv2_s feature extractor model...
[INFO] Created SummaryWriter, saving to: runs/2025-12-30/data_20_percent_without_aug/effnetv2_s/5_epochs...
100%          5/5 [00:11<00:00, 2.40s/it]
Epoch: 1 | train_loss: 1.0771 | train_acc: 0.3047 | test_loss: 0.9163 | test_acc: 0.7121
Epoch: 2 | train_loss: 0.9092 | train_acc: 0.6953 | test_loss: 0.8707 | test_acc: 0.6932
Epoch: 3 | train_loss: 0.8490 | train_acc: 0.6523 | test_loss: 0.7619 | test_acc: 0.7652
Epoch: 4 | train_loss: 0.7325 | train_acc: 0.7578 | test_loss: 0.6267 | test_acc: 0.9271
Epoch: 5 | train_loss: 0.6777 | train_acc: 0.8594 | test_loss: 0.6222 | test_acc: 0.8759
[INFO] Saving model to: models/07_effnetv2_s_data_20_percent_without_aug_5_epochs.pth
```

```
[INFO] Experiment number: 4
[INFO] Model: effnetv2_s
[INFO] DataLoader: data_20_percent_without_aug
[INFO] Number of epochs: 10
[INFO] Creating effnetv2_s feature extractor model...
[INFO] Created SummaryWriter, saving to: runs/2025-12-30/data_20_percent_without_aug/effnetv2_s/10_epochs...
100%          10/10 [00:22<00:00, 2.29s/it]
Epoch: 1 | train_loss: 1.0771 | train_acc: 0.3047 | test_loss: 0.9163 | test_acc: 0.7121
Epoch: 2 | train_loss: 0.9092 | train_acc: 0.6953 | test_loss: 0.8707 | test_acc: 0.6932
Epoch: 3 | train_loss: 0.8490 | train_acc: 0.6523 | test_loss: 0.7619 | test_acc: 0.7652
Epoch: 4 | train_loss: 0.7325 | train_acc: 0.7578 | test_loss: 0.6267 | test_acc: 0.9271
Epoch: 5 | train_loss: 0.6777 | train_acc: 0.8594 | test_loss: 0.6222 | test_acc: 0.8759
Epoch: 6 | train_loss: 0.5990 | train_acc: 0.8789 | test_loss: 0.6254 | test_acc: 0.9072
Epoch: 7 | train_loss: 0.6350 | train_acc: 0.7422 | test_loss: 0.5644 | test_acc: 0.8864
Epoch: 8 | train_loss: 0.5375 | train_acc: 0.9102 | test_loss: 0.5216 | test_acc: 0.8561
Epoch: 9 | train_loss: 0.5828 | train_acc: 0.7578 | test_loss: 0.5201 | test_acc: 0.8153
Epoch: 10 | train_loss: 0.5122 | train_acc: 0.8047 | test_loss: 0.4969 | test_acc: 0.8864
[INFO] Saving model to: models/07_effnetv2_s_data_20_percent_without_aug_10_epochs.pth
```

```
CPU times: user 25.9 s, sys: 6.93 s, total: 32.8 s
Wall time: 1min 13s
```

```
%load_ext tensorboard
%tensorboard --logdir runs
```

The tensorboard extension is already loaded. To reload it, use:

```
%reload_ext tensorboard
```

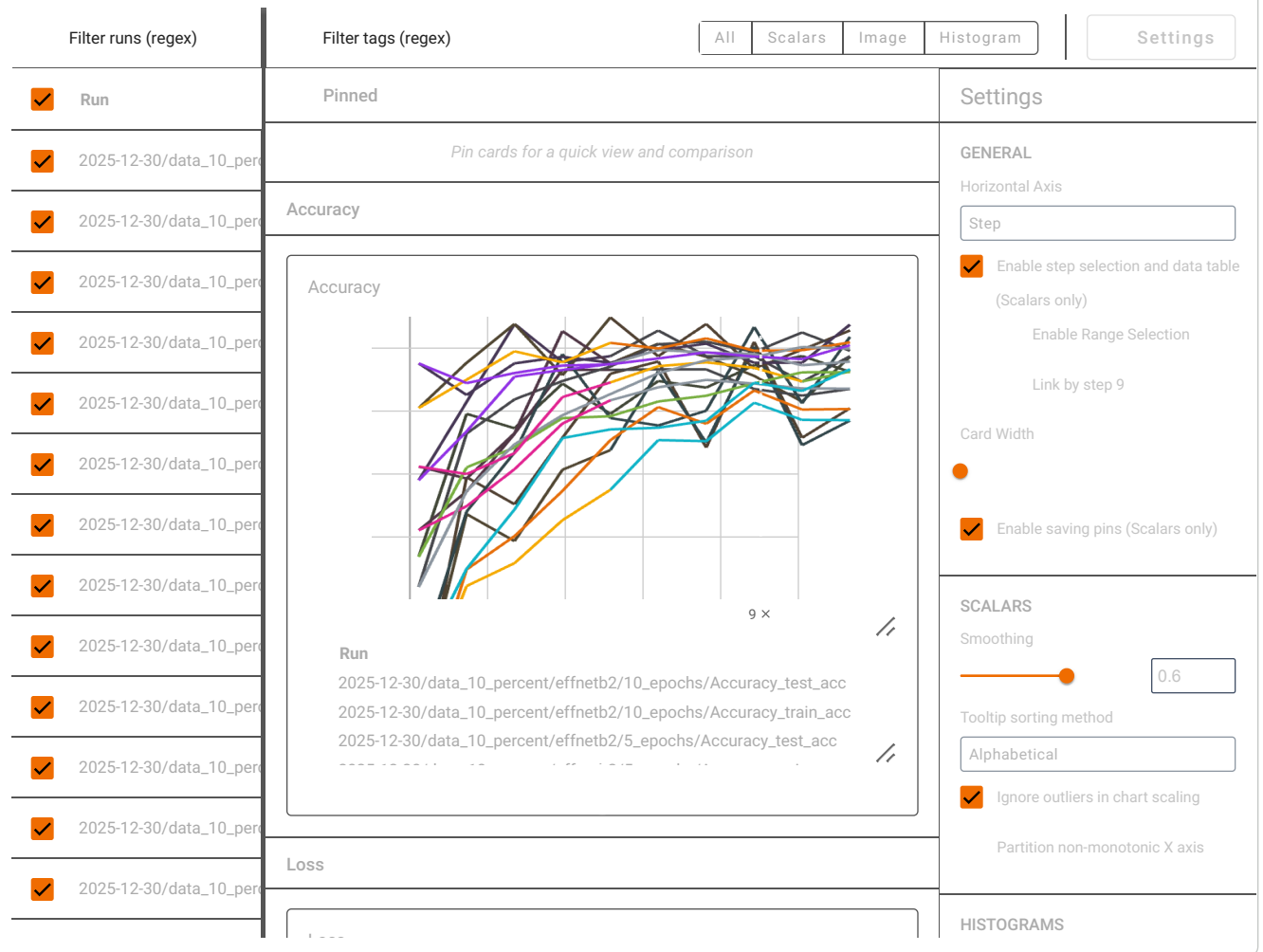
Reusing TensorBoard on port 6006 (pid 6747), started 0:01:17 ago. (Use '!kill 6747' to kill it.)

TensorBoard

TIME SERIES

SCALARS

INACTIVE



Looks like EffNetV2_S *without* data augmentation performed the best on average for test loss and test accuracy.

Let's keep going with that model for the next experiment.

Exercise 3. Scale up the dataset to turn FoodVision Mini into FoodVision Big using the entire [Food101 dataset from torchvision.models](#)

- You could take the best performing model from your various experiments or even the EffNetB2 feature extractor we created in this notebook and see how it goes fitting for 5 epochs on all of Food101.
- If you try more than one model, it would be good to have the model's results tracked.
- If you load the Food101 dataset from `torchvision.models`, you'll have to create PyTorch DataLoaders to use it in training.
- Note:** Due to the larger amount of data in Food101 compared to our pizza, steak, sushi dataset, this model will take longer to train.

We've got a model that's working (EffNetV2_S) pretty well.

So now let's turn our FoodVision Mini model into FoodVision BIG by upscaling the data.

Specifically, let's train a EffNetV2_S feature extractor on all of the Food101 data.

We'll do so for 5 epochs without data augmentation.

Download Food101 Dataset